Understanding tabular foundation models: pre-training, synthetic data

Mohit Saharan, P6, 20260420

Reference: “How Tabular Foundation Models learn without real data”, C. Molnar.

Background

- In the previous posts (P3, P4, and P5), I discussed the overview of TFMs, the hands-on code example to use them, how they make predictions, and their architecture. Today, I am going a step further to learn how these models are trained. As before, I will use the (fourth) blog post by C Molnar as a reference and make notes here as I read it.

Introduction

- This post discusses how TFMs like TabPFN and TabICL are pre-trained to enable in-context learning (=single forward pass without weight updates).
- We'll discuss the pre-training procedure and how the pre-training data are generated (also called the prior).
- The content is about general pre-training of PFMs, not a particular one, but references TabPFN and TabICL frequently.

TFMs are pre-trained on millions of tables

- TFMs like TabPFN are neural networks.
- They receive typical training ("pre-training", in this case): Large amounts of data are fed to the randomly initialized model; stochastic gradient descent adapts the model weights.
- A single "data point" represents an entire supervised ML task (a table with both training and test data) with features and the target.
  - Here, "training data" refers to the train-test split in a table, and not to the training of the TFM.
  - In subsequent text, training/test refers to the inside-table split unless stated otherwise.
- Thus, a batch of "data points" represents tables. E.g., TabPFNv2 would have 64 such tables in a batch.
  - This means 64 supervised ML tasks are "solved" per batch, and the weights of the model are updated based on how well it solves them.
- The entire pre-training of TabPFNv2 is based on 2 million batches (around 130 million tables).

The loss function to approx. the posterior predictive distribution

- The loss used is the negative log-likelihood for the target cells in the test portion of the tables.
- The loss is then computed as the expectation over the prior of datasets:

The negative log-likelihood (or cross-entropy).

We take the expectation over tables from the prior

The probability output of the TabPFN model for the target test cells

$$L_{PFN} = \mathbb{E}_{((\mathbf{X}_{test}, \mathbf{y}_{test})) \cup D_{train} \sim p(D)} \left[-\log q_\theta(\mathbf{y}_{test} \mid \mathbf{X}_{test}, D_{train}) \right]$$

- Here,
  - The loss measures the negative log-likelihood of the prediction of the target test cells.
  - To aggregate over the prior, we take the expectation of this negative log-likelihood across the datasets.
  - $p(D)$ is the prior over the datasets and is defined by the selection/creation of pre-training data.
  - To calculate this loss for stochastic gradient descent, we can replace the expectation with the sum over the 64 datasets in the batch.
- By pre-training the TFMs with this loss function, they become models that learn to approx. the PPD of y_{test}.

The loss function to approx. the posterior predictive distribution

- This loss function works for both classification and regression, which are trained separately.

Classification:

- The architecture is hard-coded with a fixed last-layer dimension (usually 10), meaning the model is natively built for a specific max number of classes.
- If a dataset has <10 classes, the model uses the first few logits and ignores the rest during pre-training.
- For >10 classes, the model uses Heirarchical Class Extensions, grouping classes into clusters and applying the model multiple times.
- This trick is applied at prediction-time, not during pre-training.

Regression:

- While pre-trained separately, it is also treated like a classification problem with the same loss function and the regression target is chunked into buckets (perhaps 100 for TabPFNv2), turning regression into a classification task.
- By predicting the buckets, we get a probability for each and can turn this into a probability distribution for the target (the predictive posterior distribution).
- This gives us multiple options for the prediction: mean, mode, median, other quantiles, uncertainty quantification.

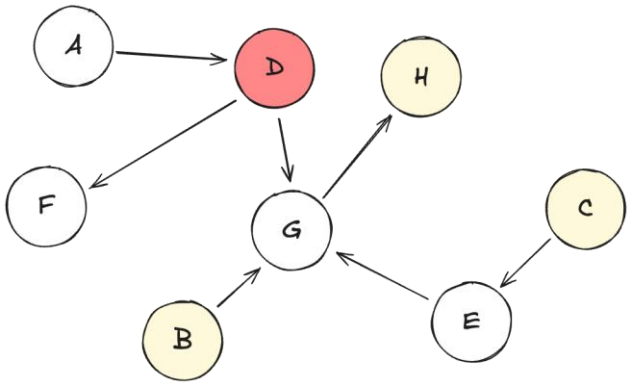
TFMs are pre-trained with synthetic data

- Pre-training a neural network to predict tabular data requires lots of tables to pre-train on.
- Many TFMs are pre-trained exclusively on synthetic data.
- General idea:
  - Design a process that spits out a great variety of synthetic data on the fly, with a train-test split, targets, & features.
  - All these synthetic datasets should show a great variety: different numbers of classes, rows, features, and different relationships between the features and targets.
  - Thus, what the prior defines here is not just a sampling procedure for tables, but for full supervised ML tasks.
- Relying on synthetic data means that TFMs are unlike typical GenAI, which was trained on copyrighted material.

Structural causal models produce pre-training data

- Causal model is at the heart of the prior for TabPFN (v1 and v2) and TabICL.
- Structured causal models (SCMs) are a framework for describing causal relationships between variables and can be used to generate data.
- To create a new table for pre-training, we first need to generate an SCM. On a very high level, it works as follows:
  - First, sample high-level parameters of the structural causal model, like the number of features, the size of the dataset, and so on.
  - Next, initiate a directed acyclic graph (DAG) that decides the relationship between features (and the target).
    - In the DAG, each node represents a variable.
    - The directed edges (arrows) represent causal influence.
  - The relationship between each node and its parents (incoming arrows) is typically set as f(Pa(c)) + ϵ, where c is the node, Pa(c) are its parents, and f is typically a sampled modeling function.
  - Not all variables (nodes) will make it into the dataset, but only a subsample will become features or targets; the others are “unobserved” variables from the perspective of the supervised ML task.

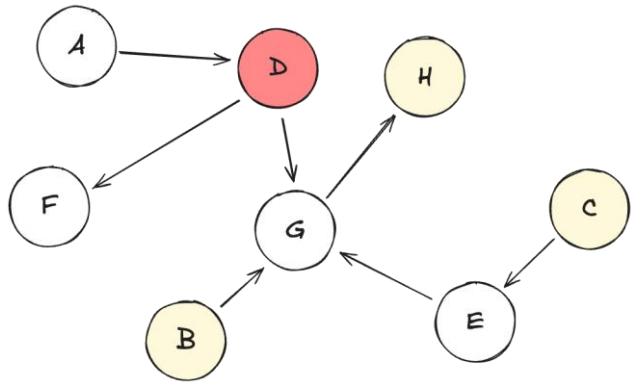
Structural causal models produce pre-training data



- This an example of a directed acrylic graph that instantiates parts of a structural causal model. Here:
- Each node from A to H is a variable.
- Variables have different sample roles:
  - D is the target.
  - B, C, and H are the features.
  - A, E, F, G won't make it onto the data ("unobserved").

Each dataset is sampled from an SCM

- Once we have an SCM fully specified, we can sample a dataset from it.
  - Reminder: this will later be just one “datapoint” for pre-training, so we have to repeat sampling another SCM, plus sampling from this SCM millions of times.
- Broadly, this is how to sample from an SCM:
  - Sample values for variables without parents. In the DAG shown (same as previously), this would be nodes: A, B, C. These are typically sampled just from Gaussian or Uniform noise.
  - Then, continue propagating values through the graph, based on the relationships described by the edges.
    - So, we would sample values for variables E and D next. These are based on the sampled functions of their parent nodes, plus typically some Gaussian noise.
  - Continue until all variables are sampled.
  - Take all the variables that are marked as features or targets and create a table from them. Mark some of the rows as training data and the others as test data.



Each dataset is sampled from an SCM

- This way, we can sample SCMs on the fly from the prior, and for each SCM, sample one dataset. We can sample 64 datasets (in case of TabPFNv2) to create a batch and continue pretraining with more batches until convergence.
- Reminder: this was a high-level overview. The TFMs use different ways to induce these SCMs: TabPFN v2 and TabICL use fully-connected multi-layer perceptrons (MLPs), while TabPFN v2 grows DAGs with a redirection sampling method.

The prior is the secret ingredient

- With tabular foundation models, we have a new “engine”, the PFN, that learns how to do in-context learning without any hyperparameter tuning needed to outperform Catboost and other ML algorithms on many supervised ML tasks.
- The fuel to the engine is the prior, the process of sampling data for pre-training.
- With the design of the prior, a lot of assumptions are now poured into the model. Or rather, it decides what the model learns. Here are some choices that the author finds interesting:
  - Structural causal models seem to work pretty well, so basing the prior on causality seems to pay off.
  - TabICL additionally uses XGBoost in the SCM edges trained on noise variables, since tree-based models seem to carry the right inductive biases for tabular data.
- And since TFMs are neural networks, they can also be continued to be pre-trained, or fine-tuned. A few examples:
  - Models like TealTabPFN v2.5 and TabDPT included real-world data in their pre-training.
  - TabPFN-Wide continues pre-training on synthetic data.
  - CausalFM pre-trains on causality-inspird Bayesian neural networks to handle causal inference stuff like Conditional Average Treatment Effect (CATE) estimation.

Outro

- The architecture of TFMs is expected to continuously improve over the next few years to become faster at prediction time (currently very slow) and handle more table cells (features and rows).
- However, capability-wise, the prior seems to be where the leverage is:
  - If one wants a model that works well with their highly specific company's tabular data, they could fine-tune a TFM model on internal data.
  - If one wants a TFM that works well for data with repeated measurements, they could define a data generating process that spits out clustered data.